



AIRSDN: AI based routing in software-defined networks for multimedia traffic transmission

Anıl Dursun İpek^a, Murtaza Cicioğlu^{a,*}, Ali Çalhan^b

^a Department of Computer Engineering, Bursa Uludağ University, 16059, Bursa, Türkiye

^b Department of Computer Engineering, Düzce University, 81620, Düzce, Türkiye

ARTICLE INFO

Keywords:

SDN
Machine learning
Routing
Quality of service (QoS)

ABSTRACT

With the rapid increase in internet usage and the number of network-connected devices, network management and optimization have become increasingly challenging, particularly for high-bandwidth applications such as video streaming. The decentralized structure of traditional networks and the lack of standardization further complicate these challenges. Software Defined Networking (SDN) has emerged as a solution, enabling a more flexible and programmable architecture by centralizing network control. However, existing SDN controllers typically determine the optimal path based on simple metrics such as hop count and bandwidth, which can be insufficient in high-traffic scenarios. To overcome these limitations, this study proposes a novel artificial intelligence (AI)-based routing algorithm. Operating within the SDN framework, the proposed algorithm analyzes network traffic levels and dynamically selects the most efficient data transmission paths. The proposed algorithm is simulated in Mininet, a virtual network environment, using a network model inspired by real-world internet structures (NSFNET). Simulations are conducted under varying traffic conditions, with TCP (Transport Control Protocol) data and video transmission scenarios. Key performance metrics are observed, including round-trip time (RTT), throughput, packet loss, and video quality (measured using PSNR and SSIM). The machine learning model was trained using a custom dataset consisting of 876 records generated in the Mininet environment. Although the dataset size is sufficient for the simulation environment, caution should be exercised when generalizing the results to real-world network conditions. Future studies may aim to enhance the model's reliability by exploring data augmentation techniques and utilizing larger datasets that include real-world data. To classify traffic levels, machine learning models are trained, and the best-performing model (Logistic Regression) is integrated into the proposed routing algorithm. The results demonstrate that the proposed AI-based routing algorithm significantly improves network performance compared to both traditional hop-count-based and QoS-aware routing. Particularly in high-traffic scenarios, it achieves lower latency, higher throughput, and better video quality. Additionally, resource usage was analyzed on a Raspberry Pi 5 device, revealing stable RAM consumption (~50 %) and fluctuating CPU utilization (10–90 %), indicating the feasibility of lightweight deployment with awareness of processing load. This study highlights the potential of AI-driven SDN frameworks for adaptive and efficient network traffic management in high-demand applications, offering a robust solution for dynamic routing.

ACRONYMS

SDN	Software-Defined Networking
AI	Artificial Intelligence
TCP	Transport Control Protocol
UDP	User Datagram Protocol
RTT	Round Trip Time
PSNR	Peak Signal-to-Noise Ratio

(continued on next column)

(continued)

SSIM	Structural Similarity Index Measure
QoS	Quality of Service
IoT	Internet of Things
ECMP	Equal-Cost Multipath
LR	Logistic Regression
KNN	K-Nearest Neighbors
SVM	Support Vector Machine

(continued on next page)

* Corresponding author.

E-mail address: murtazacicioglu@uludag.edu.tr (M. Cicioğlu).

<https://doi.org/10.1016/j.comcom.2025.108222>

Received 23 January 2025; Received in revised form 21 April 2025; Accepted 21 May 2025

Available online 25 May 2025

0140-3664/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

(continued)

ANN	Artificial Neural Networks
ML	Machine Learning
DL	Deep Learning
ACO	Ant Colony Optimization
PSO	Particle Swarm Optimization
eMBB	Enhanced Mobile Broadband
CDN	Content Delivery Network
NFV	Network Functions Virtualization
NSFNET	National Science Foundation Network
IPOP	Interconnected Access Point
IP	Internet Protocol
IPv4	Internet Protocol version 4
API	Application Programming Interface
REST	Representational State Transfer
NETCONF	Network Configuration Protocol
YANG	Yet Another Next Generation
MSE	Mean Square Error
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
bps	Bits per second
DDOS	Distributed Denial of Service
ODL	OpenDaylight
ONOS	Open Network Operating System

1. Introduction

The use of computer networks is becoming increasingly common as the number of devices on the network increases exponentially. According to the 2024 Global Internet Phenomena Report by Sandvine [1], video content constitutes the largest share of downstream internet traffic across both fixed and mobile networks, accounting for 38 % of the total. This dominance underscores the significant impact of video-based data on global internet usage. As a result of these developments, traffic on the network is gradually increasing and this trend is expected to continue with further technological developments. This increase in connected devices poses significant challenges in managing traditional network systems that often rely on built-in management protocols within each device [2–4].

Network devices operate on two distinct planes: the data plane and the control plane. The network nodes make routing decisions and create communication paths for data packets in the control plane. The data plane is responsible for physically transmitting these packets over the network. The main challenge in traditional network systems is that each network device makes and implements its own routing decisions independently of the others. The lack of centralized control, compounded by the proprietary protocols employed by different vendors, hinders the development of a universal communication language. Consequently, integrating new functionalities and network updates becomes a complex and vendor-specific task, ultimately increasing management costs [5–9].

The idea of software-defined network (SDN) was put forward in studies conducted at Stanford University in 2008 to solve the problems of existing traditional networks. The basic principle of these networks is to address the scalability, management, flexibility, and cost problems in traditional networks and to bring an innovative approach. In SDN architecture, control, and data planes are separated for creating a centralized approach. The controller in the control plane is the brain of the network and is responsible for managing the network. Controllers are computers/servers with high processing power and make general decisions on the network. Switches, routers, and other intermediate devices (such as middlebox) are in the data plane and they implement the decisions taken in the control plane and are only physically involved in packet transmission [10–13].

The emergence of SDN has revolutionized network management by enabling a holistic view of the network topology. This paradigm shift empowers network programmability, fostering increased efficiency and optimization. Improving resource utilization and increasing quality of

service (QoS) are needs of future networks. When the service quality of traditional networks is examined in areas such as data centers and content distribution networks, it is seen that they are inadequate due to their lack of a global view. While traditional networks employ load balancing and routing protocols to manage the QoS and optimize network performance, research has demonstrated that these methods are demonstrably less efficient comparing their SDN counterparts [11–16].

The load balancing and routing algorithms have similar functions. They have basic tasks such as minimizing response time on the network, optimizing resource usage, avoiding bottlenecks, and maximizing efficiency [17]. This study proposes an AI-based routing algorithm for data transfer between the server and the client in a virtual environment. In the simulation environment, SDN controllers select the data transmission path by making decisions based on limited information, typically using a single parameter such as hop count or latency during the routing. More reliable data transmission is made possible by the controller's centralized function, which enables the acquisition of certain parameters during data transmission between two network devices. However, as the number of parameters used in the routing increases, the decision-making mechanisms must also be optimized. In this study, a multi-parameter AI-based algorithm is proposed to optimize the routing process for data transmission between the server and client by leveraging the controller's powerful capabilities. The flexible and centralized structure of SDN is used in the design of the algorithm. It is done by dynamically evaluating the general view of the network with AI models. This study aims to reduce network traffic, optimize bandwidth usage, and improve user experience.

AI is an approach that aims to provide human-like intelligence features to computer systems [18–20]. AI is accelerating at an unprecedented pace, powered by the exponential growth of computing power, vast datasets, and advances in subfields such as machine learning and deep learning. In network management, AI exerts a transformative influence across a diverse spectrum of applications. Smart home systems create smarter networks by communicating with automated vehicles and other Internet of Things (IoT) devices [21]. It plays an important role in network security in detecting, analyzing, and preventing attacks [22]. AI algorithms are adept at detecting potential security breaches by analyzing network traffic for abnormal patterns such as unusual traffic volumes, unauthorized access attempts, or suspicious data packets, thereby increasing overall network resiliency. It also optimizes network performance, manages data traffic, increases transmission speed, or contributes to more efficient operation of networks by regulating energy use. Heuristic algorithms, informed by AI decision-making models, can be incorporated into routing protocols to enable dynamic traffic management on the network [23,24].

Floodlight [25] controller and Mininet [26] are used as the development environment in this study. Mininet is a simulation tool that provides a virtual network development environment and offers Python API support. Floodlight is an SDN controller being developed in Java. Floodlight offers a language-independent development environment between the network controller and the programmer with the REST API. The iperf3 tool is used to generate traffic during the development phase of the study. Therefore, UDP and TCP traffic can be created on the network. To measure user QoS in the network, video is transferred with the ffmpeg tool. After the transfer, the initial version of the video is compared and as a result, PSNR (Peak Signal-to-Noise Ratio) and SSIM (Structural SIMilarity Index) values are produced. The change in video quality is evaluated with the data obtained.

The study offers several significant contributions to network management utilizing AI. Firstly, it enhances user experience by facilitating lower latency data transfers and enabling the delivery of higher-quality content, such as high-resolution video streams. Secondly, the model, trained on a comprehensive dataset specifically created within a virtual simulation environment using Mininet, dynamically optimizes network traffic by making real-time routing decisions. This approach fosters

improved network performance and efficiency. Thirdly, the study promotes efficient resource utilization by leveraging AI for intelligent resource allocation. By distributing the load equally across available data paths, the model prevents unnecessary resource consumption and optimizes network operations. Finally, the study contributes to the advancement of the field by creating a novel dataset within the Mininet testbed, which can serve as a valuable resource for future research endeavors. This manuscript is an extended version of the conference paper presented in [27]. In addition to the main contributions, our study contains various difficulties in the implementation phase. These challenges are:

- Implementing the application within a virtual environment presented a unique challenge. Due to the absence of physical distances between network devices in Mininet, routing decisions solely based on hop counts introduced complexities in the application logic.
- The model's training relied heavily on a suitable dataset for the virtual environment. The lack of relevant datasets in existing literature necessitated the creation of a custom dataset within the Mininet testbed. This highlights the importance of domain-specific datasets for achieving optimal performance in machine learning and deep learning applications.
- While the controller efficiently handled tasks like traffic prediction and switch communication during routing, neglecting its CPU consumption could pose scalability limitations in the future. Our observations indicated a gradual increase in controller CPU utilization (mention specific percentage or processing time increase if measured). Implementing resource management strategies or exploring alternative controller architectures could be potential areas for future investigation.
- Compared to the theoretical studies, limited practical applications brought another difficulty. This gap emphasizes the need for more research efforts focused on the real-world application of AI-powered network management solutions.
- Finally, collecting network information from a limited number of simulated tools within the constraints of the simulation environment resulted in disregarding the time factor during data collection. The main reason for this problem is that the simulation environment has certain restrictions.

This study investigates the application of AI for dynamic traffic routing in SDN. We leverage the Mininet framework to emulate the NSFNET topology and implement a novel AI-based routing algorithm. This algorithm optimizes data transfer paths between designated clients and servers, prioritizing low-traffic routes to enhance user experience. The performance of our proposed AI-based routing is compared with the default hop-based routing and the QoS-aware routing of the Floodlight controller based on efficiency and latency parameters.

1.1. Motivation and contributions of the study

Traditional networks face significant challenges in network control and optimization due to their decentralized management, lack of standardization, and complex infrastructure. The increasing number of network devices and the resulting surge in traffic volume further exacerbate these issues. SDN aims to address these challenges by introducing an innovative solution that separates the control and data planes, enabling centralized management. The centralized, programmable, and flexible architecture of SDN provides a more holistic approach to network management and enhances efficiency.

However, existing SDN controllers (e.g., Floodlight, Ryu) often rely on basic metrics, such as hop count, for routing decisions. This approach may be insufficient, especially for applications that require high bandwidth, such as video streaming. As a result, network performance and user experience may not meet the desired levels, necessitating more advanced routing algorithms that consider real-time network

conditions.

The rapid advancements in artificial intelligence (AI) techniques in network management present significant opportunities for optimizing network performance and making more intelligent decisions. AI algorithms can analyze network traffic, detect anomalies, improve network efficiency, and optimize resource utilization. In this context, AI-based routing algorithms have the potential to dynamically evaluate network traffic congestion and select the most optimal data transmission paths.

The primary motivation of this study is to overcome the limitations of existing SDN controllers that rely on simplistic routing metrics and to develop a more adaptive and efficient network traffic management solution for high-demand applications. In line with this motivation, the study makes the following key contributions:

- A novel AI-based routing algorithm is proposed within the SDN framework, which dynamically selects the most suitable data transmission paths by considering real-time network traffic conditions.
- The proposed algorithm is simulated in a virtual environment (Mininet) inspired by real network topologies (NSFNET), and its performance is evaluated under different traffic conditions.
- A unique dataset is created to classify traffic levels, and various machine learning models are trained on this dataset to identify the best-performing model (Logistic Regression).
- The best-performing machine learning model is integrated into the proposed AI-based routing algorithm, enabling dynamic routing decisions based on traffic levels.
- Simulation results demonstrate that the proposed AI-based algorithm significantly improves key network performance metrics, such as round-trip time (RTT) and throughput, compared with the default hop-based routing and the QoS-aware routing of the Floodlight controller. Additionally, video transmission scenarios show a substantial enhancement in video quality, measured using PSNR and SSIM values.
- The study highlights that AI-driven SDN frameworks offer a more flexible, efficient, and user-centric solution for high-demand applications.
- The dataset generated in the Mininet environment serves as a valuable resource for future research.

The results, presented graphically, demonstrate significant improvements in network performance achieved through our AI-driven routing strategy. Section 2 summarizes the related works and research conducted in the literature. In Section 3, the tools and development environment used are explained. The materials and methods are thoroughly explained in Section 4, followed by the presentation of results in Section 5.

2. Related works

In his study, Ahmet W [28]. discusses the Equal-Cost Multipath (ECMP) routing algorithm, a traditional load balancing method, along with the bandwidth load balancer he developed. ECMP is statically run on the network and evenly distributes incoming streams to the next network devices. However, its major disadvantage is its static nature, as it is unaware of the overall state of the network. In contrast, the bandwidth load balancer dynamically transfers streams based on bandwidth statistics received by the controller, according to the approach presented in the study. This approach enables switches to be aware of the overall network state and direct flows toward less congested paths. The algorithms are evaluated within a topology designed using the Floodlight controller in the Mininet environment. According to the results, the proposed approach increases network throughput by 30 %. However, the proposed approach also faces some challenges. The tests are conducted on a small topology, so the algorithm needs further development to be integrated into larger networks. Additionally, the tests neglected

the CPU load on the controller, which is a known issue as the processing capacity of the controller is limited and may cause delays in resource usage situations.

Bilal B. and Banu U [29]. investigate load balancing in data centers using AI techniques. Traditional load-balancing methods are deemed insufficient due to the intensity of data center use and the rapid increase in data volume. SDNs offer dynamic load balancing as an alternative to traditional methods in data centers. The study focuses on utilizing AI techniques such as Artificial Neural Networks (ANN), Support Vector Machines (SVM), logistic regression (LR) for load balancing. The goal is to reduce delays between servers and enhance overall system performance. A dataset was generated by performing tests on the system topology implemented with SDN. This dataset comprises 800 training data points and 192 test data points. The most successful model identified through training is ANN. The integration of AI into the system has resulted in improved performance.

Ahmed H. and Ahmadreza M [30]. examine AI-based load balancers used in SDN from a broad perspective. First of all, it divides load balancers into four classes. These categories include nature-inspired load balancers, machine learning-based load balancers, mathematical model-based load balancers, and other types of load balancers. Under each heading, studies conducted in the literature after 2017 were examined in detail. A comparison table is created to highlight the advantageous and disadvantageous parts of the research. The parameters used for load balancing are listed according to their frequency of use and presented as a reference for future studies. According to the research, the five most used network parameters for load balancing are throughput, response time, latency, workload degree, and packet loss ratio. The study emphasizes that previous studies did not use sufficient QoS parameters and provides a good perspective for future studies.

Mohamed I. Hamed et al.'s study [31] discusses the problems associated with traditional load balancers, such as lack of flexibility and difficulties in managing network flows. The article proposes the use of SDN as a solution to these problems. Traditional load balancers are inflexible and have been criticized for high hardware costs. SDN is suggested as a solution because it offers centralized control and programmability, which can address these issues. The study introduces a bandwidth-based load-balancing approach that distributes network demands among servers based on bandwidth consumption. The performance of the proposed approach is evaluated against Round Robin and Links-based load balancing schemes in the Mininet simulation environment and Raspberry Pi application. The results indicate that the proposed approach outperforms these traditional schemes. Overall, the study concludes that SDN can provide an economical and flexible solution to the problems associated with traditional load balancers.

In the studies of Mosab H. et al. [14] wrote an extensive research paper on load-balancing techniques. They begin by discussing the advantages of SDN over traditional networks. Then, they delve into the basic objectives of load balancers, which include minimizing response time, optimizing resource usage, avoiding bottlenecks, and maximizing throughput. Load balancers are categorized as data and control plane load balancers. The data plane is further divided into server and connection load balancers, while the control plane is subdivided into logically centralized, physically distributed, and virtualized load balancers. Research findings indicate that SDN-based load balancers outperform traditional load balancers. Additionally, the paper summarizes the network parameters used in load-balancing methods and discusses security issues in SDN.

In the study by Mohammad R. B. et al. [23], the authors discuss how the increase in service demands from cloud users significantly raises the density and load on the network. This reduction in efficiency leads to system slowdowns. To improve system efficiency, it must be capable of handling various loads and increasing needs, and show dependencies on automatic measurement systems. The research presented examines two Metaheuristic optimization techniques, Ant Colony Optimization (ACO) and Particle Swarm Optimization (PSO), to investigate how these

techniques can effectively solve load-balancing problems in SDN. An SDN network topology of six switches and eight hosts was used in an experiment to compare the performance of ACO and PSO. Through the analysis, it is observed that PSO has lower latency and better performance. The study demonstrates a method to increase performance and improve load-balancing capabilities through modifications to existing optimization techniques. As a result, the study proposes to evaluate node and link reliability in SDN.

Ekber Ç. K. and Müge E. Ö [32]. propose an architecture with a new routing algorithm based on the user access protocol for the eMBB (Enhanced Mobile Broadband) content of the 5G network. They have carried out their work using the ONOS controller in the Mininet environment. In the SDN topology, 5 servers, 13 network devices and 4 users were used to generate traffic. One of the servers is located very close to the receiver, but there is TCP or UDP traffic on the path between the receiver and the server. The other server is in a traffic-free environment, but there is a greater distance between it and the receiver. In their study, the receiver requests video content from the servers. The change in video quality is observed by comparing the transferred videos before and after transmission. The quality parameter used at this stage is PSNR. The Dijkstra algorithm, which finds the shortest distance, has been used. The researcher compares the algorithm that routes according to traffic density with traditional DNS-based routing, which considers the number of hops in the topology, and concludes that it gives a 60 % better PSNR value.

In their study, Ayşe N. T. and Mevlüt E [33]. classify security threats occurring in SDN with machine learning algorithms. Attacks were carried out by sending DDOS attacks in the topology created on Mininet. Incoming attack packets were classified as either threat (1) or no threat (0). A dataset was created through tests on the existing network, and a mixed dataset was obtained by combining this dataset with another. ANN, Random Forest, XGBoost, AdaBoost, and GradientBoost techniques were used in classification processes, and performance comparisons were made between these techniques. It has been determined that the performance of the algorithms is mostly high, and Random Forest has the highest accuracy rate on the dataset.

In their study, Peter B. et al. [34] research to ensure equal distribution of the load between proxy servers in the content distribution network (CDN). They developed an AI model utilizing the flexible structure of SDN and their ability to understand the general state of the network. The research involved obtaining bandwidth, number of hops, and other network parameters along the path, which were then used as input for the AI model. The data obtained was trained with Naive Bayes, Decision Tree, Random Forest, Logistic Regression, and K-nearest neighbor algorithms. In the training results, the Decision Tree algorithm is more successful than other algorithms and can make predictions with a 97 % accuracy rate. According to the research results, the classical nearest server approach is compared with the AI-supported load-balancing system, and the positive effects of AI on the system are observed.

Thabo S. et al. [35] present a comprehensive survey of load-balancing methods. According to their research, they examine load balancing under five main headings. These are controller load balancing, server load balancing, load balancing on wireless connections, communication path selection load balancing, and AI-based load balancing. In these subheadings, studies conducted in the general literature are discussed, and the weaknesses and strengths of the studies are argued. In addition, at the end of the research, the performance criteria and application tools used in the projects were examined. In research, 93 % of the studies use delay, 87 % throughput, 100 % QoS, and 73 % complexity parameters. When the application tools are examined, 55 % of the researchers test their proposed approaches in the Mininet environment. The most commonly used tool to generate network traffic is iperf3.

Cui C. and Xu Y [36]. aim to distribute the load equally among different subnets by taking advantage of the global view of SDN. In their study, an ANN is developed using bandwidth usage rate, packet loss rate,

transmission delay, and hop count. The developed AI model is compared with the static round-robin algorithm. The results show that the network delay decreases by 19.3 % at most, and it is determined that there is an improvement in the system.

Şükran D. and Derya Y. [37], in their study utilizing the Floodlight controller as an SDN controller, processed the statistics obtained from packet monitoring on network traffic using MATLAB. The paper employs optimization techniques such as linear search, tabu search, and simulated annealing. The analysis indicates that the traditional linear search method outperforms tabu search; however, the proposed hybrid algorithm (simulated annealing-tabu search) yields more efficient results. It is observed that while Metaheuristic techniques do not always guarantee the best performance, they offer significant improvements under suitable conditions. The study also leverages AI to predict packet flow in SDN and conducts tests on different network topologies to identify the topology with the highest accuracy.

Anteneh A. Gebremariam et al. [38] focused on using machine learning to solve complex problems in SDN and Network Functions Virtualization (NFV) based networks. The study examines the use of AI in five main areas of SDN and NFV-based networks: network architecture, load balancing and resource utilization, fault detection and management, network management and operations, and network security. While the centralized structure of SDN and NFV offers flexibility in network management, it also introduces significant security risks if the central controller is compromised. The authors emphasize that machine learning can enhance network resilience in such environments by creating self-adaptive and self-managing systems. However, they also note that research in this field is limited and faces various challenges, highlighting the need for more in-depth studies to better understand and harness AI's potential in these networks. Table 1 lists the aims of the studies and the simulation programs reviewed in the literature. Some researchers did not perform simulations but only analyzed the data with different analysis methods.

3. Machine learning-based routing approach in SDN-based architectures

SDNs are defined by a three-tier architecture consisting of the infrastructure (data), the control, and the application planes. There is a southbound interface between the data and control planes. The southbound interface most commonly uses the OpenFlow protocol. This protocol establishes communication between the switches and the controller. According to the protocol, when packet flows on the network arrive at the switch, it first checks its flow table. If the information about the packet is not in its flow table, it contacts the controller on the network and asks the controller to define a new rule. After the controller decides on the packet status, it updates the flow table of the relevant switch. The protocol is generally based on packet transmission according to the match-action status in the flow tables. There is a northbound interface between the control and application plane. The northbound interface provides communication between the programmer and the controller for programming. Therefore, the programmer can have an overview of the network, check the instant status of the devices in the network, and set new rules. Although not a de facto standard [9], protocols such as REST, NETCONF, and YANG are commonly used for the northbound interface in SDN.

Today, data analysis and machine learning play critical roles in many industries. When the complexity and volume of datasets increase, effective modeling and accurate prediction-making become increasingly important. Thus, machine learning algorithms are used effectively, especially in classification tasks. In this study, the collected data are analyzed to identify the features that influence traffic levels. Subsequently, machine learning models are trained using these identified features. Training results and data analysis are shared in the research results section.

The proposed routing algorithm is run before video transfer between

Table 1

Summary of some relevant studies we reviewed.

Author	Concept	Finding	Disadvantages
Warsama [28]	Creating a new bandwidth-based routing algorithm and comparing it with the classical ECMP algorithm.	The algorithm possesses global network knowledge and directs data flow in a non-congested direction.	Neglecting the CPU load and testing the algorithm on a small topology.
Bilal and Banu [29]	Research on load balancing in SDN-based data centers using AI techniques.	Enabling dynamic load balancing with SDN and enhancing system performance with AI.	The scarcity of deep learning applications for network issues and insufficient data.
Ahmed and Ahmadreza [30]	A comprehensive investigation of AI-based load balancing in SDN.	A comprehensive presentation of techniques and variables used since 2017.	In the examined studies, only a subset of QoS parameters are taken into account, while other criteria are overlooked.
Mohamed et al. [31]	Server-based load balancing using SDN.	Achieving better results with a bandwidth-based approach compared to classical load balancing algorithms. Testing the algorithm in both virtual and real environments.	Testing was conducted on very small topologies and showed a decline in performance after a certain threshold value.
Mosab et al. [14]	A comprehensive survey of load balancing techniques in SDN.	Demonstrating that SDN-based load balancers are more successful than traditional load balancers and examining their differences.	The studies reviewed indicate security and scalability issues in the control plane.
Mohammad et al. [23]	AI based reliable load balancing framework in SDN.	Enhancing performance and enabling load balancing through modifications to existing AI optimization techniques.	Algorithms are not being tested in large topologies and outside virtual environments.
Ekber [32]	Design of a controller based on an access protocol for eMBB traffic in Software-Defined Content Delivery Networks.	Development of a more successful routing algorithm based on the user access protocol in SDN and explanation of the results using the PSNR metric.	Not considering CDN cache management and not comparing it with existing control algorithms.
Ayşe and Mevlüt [33]	Classification of security threats in SDN using ML algorithms.	Enabling the automated detection of threats using AI.	Insufficient explanation of the datasets used in training ML algorithms.
Peter et al. [34]	Classification-based load balancing in content delivery networks.	Developing an AI model with a 97 % success rate by obtaining bandwidth, hop count, and other network parameters along the path and observing positive effects in the system.	Insufficient explanation of the dataset used in training the models.
Thabo et al. [35]	Intelligent load balancing techniques in SDN: a survey.	A comprehensive examination of load balancing in SDN and identification of the most used	It provides a general approach to AI-based load-balancing techniques. It is a

(continued on next page)

Table 1 (continued)

Author	Concept	Finding	Disadvantages
		network parameters and technologies.	suitable article to gain information about algorithms and studies. Details can be learned by examining the mentioned studies.
Cui and Xu [36]	Development of an AI-based load-balancing algorithm in SDN.	Developing an artificial neural network using bandwidth utilization rate, packet loss rate, transmission delay, and hop count to improve the system.	Insufficient explanation of the dataset used in the training phase.
Sukran and Derya [37]	SDN application with AI techniques.	Monitoring and optimizing network traffic using AI techniques.	AI techniques do not always deliver the best performance.
Anteneh [38]	Applications of AI in the SDN and NFV	A brief research presentation presents the main application areas of ML in SDN and NFV-based networks.	The limited research in the field and the potential for major security vulnerabilities due to possible errors.

the client and the server. In the first stage, the algorithm loads the AI model and prepares to make predictions. After the AI model is loaded into the system, the five shortest paths between the client and server are taken from the Floodlight controller. Each path is set with StaticEntryPusher, the necessary switches are added to the flow table, and the flow statistics between the two devices on the specified path are collected. The collected statistics are given as input to the AI model, and a resulting traffic level is obtained. The resulting traffic levels are recorded in a dictionary with the route index. After the same process is performed for all routes, the route with the lowest traffic level is selected. By updating the flow table of the relevant switches on the selected route, video transfer between the client and server is ensured over that route. In addition, to address potential changes in network traffic conditions, the algorithm is periodically re-executed at pre-defined intervals during video transmission. This allows it to dynamically reassess traffic levels and, if necessary, update the selected route to avoid congestion and maintain optimal transmission performance. The pseudocode of the proposed routing algorithm is given in Table 2. The NSFNET topology and scenario used in this study are shown in Fig. 1.

Table 2
AI-based routing algorithm pseudocode.

Algorithm:	AI-based routing algorithm
Input:	Client and server IPv4 addresses
Output:	Setting the path with the lowest traffic level between the client and server
1:	function RoutingAlgorithm(client_ipv4, server_ipv4)
2:	model ← LoadTheClassificationModel()
3:	paths ← GetTheShortestPaths(client_ipv4, server_ipv4)
4:	pathsWithTraffic ← Declare a new array
5:	for path in paths do
6:	AddToFlowTable(path)
7:	pathStatistics ← GetThePathStatistics()
8:	prediction ← model.Prediction(pathStatistics)
9:	pathsWithTraffic.Append(prediction)
10:	DeleteAddedFlows(path)
11:	end for
12:	path ← FindThePathWithLowestTraffic(pathsWithTraffic)
13:	AddToFlowTable(path)
14:	return path
15:	end function

This study's development environment utilized the Ubuntu 20.04 operating system on a machine with an Intel I7 quad-core processor and 8 GB RAM, leveraging Floodlight as the SDN controller, Mininet for simulation, Iperf3, NFStream, and Ping for traffic analysis, Google Colab for the machine learning platform, and Python3 as the programming language.

Mininet is an open-source emulation and testing platform for network protocols and applications. 55 % of the studies in the literature on load-balancing algorithms in SDNs are done in the Mininet environment [35]. Mininet allows us to create virtual network topologies and simulate network protocols without physically using real network hardware. Since the study is conducted in a virtual network environment without requiring physical devices, development and testing costs are significantly reduced. Some basic topology types to create in Mininet are Single Switch, Linear, Tree, and Full Mesh. Additionally, the Python API allows the creation of the desired topology on Mininet. Any IP controller can be integrated into the created topology and managed with the help of the controller [26].

Floodlight, an open-source SDN controller, empowers network administrators with dynamic resource management and programmability. Leveraging the OpenFlow protocol, it controls network traffic, manages switches, and offers a REST API for developers to create flexible and scalable SDN applications in any programming language [25].

The National Science Foundation Network (NSFNET), which operated from 1985 to 1995, served as a vital research and education network within the United States. Its core network and interconnected access points (IPOPs) facilitated high-speed data transmission between universities, research institutions, and government agencies. Notably, NSFNET played a critical role in Internet development by utilizing and standardizing the TCP/IP protocol, laying the foundation for the broader Internet infrastructure that emerged in 1995 [39,40]. In this study, different scenarios are developed by leveraging the complex topology of NSFNET. The presence of alternative paths between different network devices and the variation in hop counts between these paths are key factors in designing the scenarios. NSFNET is chosen due to its sufficient network complexity and recognition in the literature.

To enhance the realism of the network environment, we continuously generate traffic between clients using iperf3. This versatile tool, commonly used for network performance testing, acts as a server-client model to simulate data transmission and measure key metrics like throughput, latency, and packet loss. In our study, different clients are positioned on the path between the server and the client. TCP packet transfer is carried out between these clients. The primary reason for utilizing TCP is its ability to provide reliable data transmission and ensure that data packets are correctly received by the destination. Through its congestion and flow control mechanisms, TCP minimizes losses and errors during data transmission. This allows us to realistically evaluate the overall performance of the network and accurately simulate challenges such as potential delays and packet loss. Additionally, during TCP measurements conducted with the iperf3 tool, critical information such as the amount of data transferred and bandwidth for each transmission interval is obtained. These details are crucial for assessing network performance and enabling a more detailed analysis of the network's data transmission capacity, latency, and efficiency. An example topology is shared in Fig. 2.

3.1. Video transfer between server and client

FFmpeg is an open-source multimedia framework. It provides tools and libraries for encoding, decoding, converting, and publishing audio, video, subtitles, and other multimedia files. FFmpeg supports many different multimedia formats and has a large user base. This framework contains a set of commands available via the command line. In the tests performed in our study, a video with.mp4 extension is converted to.ts (Transport Stream) extension with the “ffmpeg -i video.ts output.ts” command of the FFmpeg library. TS is a transport format that contains

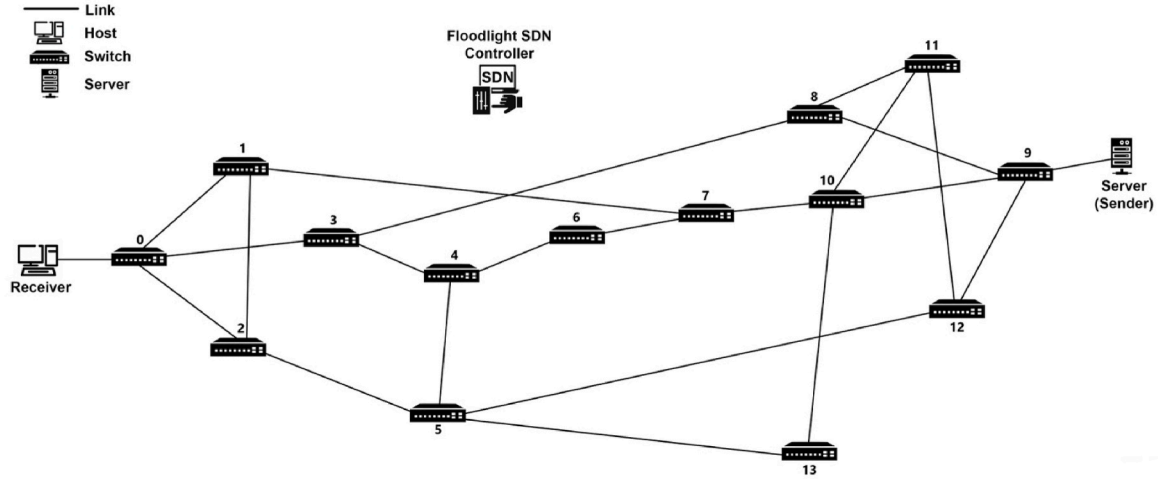


Fig. 1. NSFNET topology implemented in the Mininet.

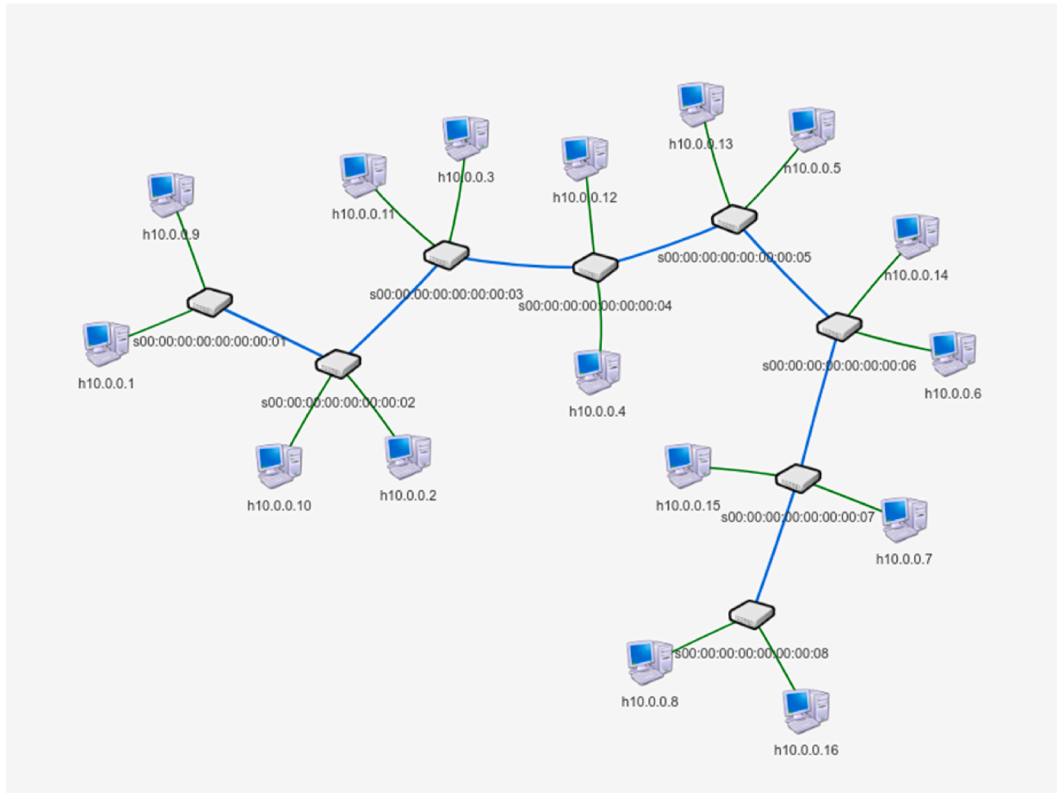


Fig. 2. Example linear topology in Mininet environment.

video and audio data. While the number of frames sent from the original video (video.ts) is 3990, the number of frames of the video received by the server (input.ts) is 3881. 9 frames were lost during transmission. The representation of normal and corrupted frames in the video obtained after the transmission is provided in Fig. 3.

NFStream is a Python framework designed to make working with network data easy and intuitive [41]. It offers the opportunity to analyze data on the network practically. In our study, an interface on the network is monitored using NFStream to collect statistical data on packets. It creates a copy of existing packets while listening and does not interfere with the network. In the AI model, we have established that 33 features are obtained with NFStream, and the effects of these features on the traffic level in the network are examined. In our study, when

creating the NFSteamer object, the `idle_timeout` property was set to 15 s, and `statistical_analysis` was set to True. Other features are used by default. The system flowchart developed while designing the proposed architecture is shown in Fig. 4.

3.2. Creating video streaming scenarios on the NSFNET topology

Certain scenarios have been created to obtain data on the NSFNET established in the Mininet environment. The scenarios are based on data transmission on three different paths on the network. Each path contains a different number of hops, and this variation forms the foundation of the scenarios within the simulation environment. The scenarios are structured based on changes in hop count, established connections, and

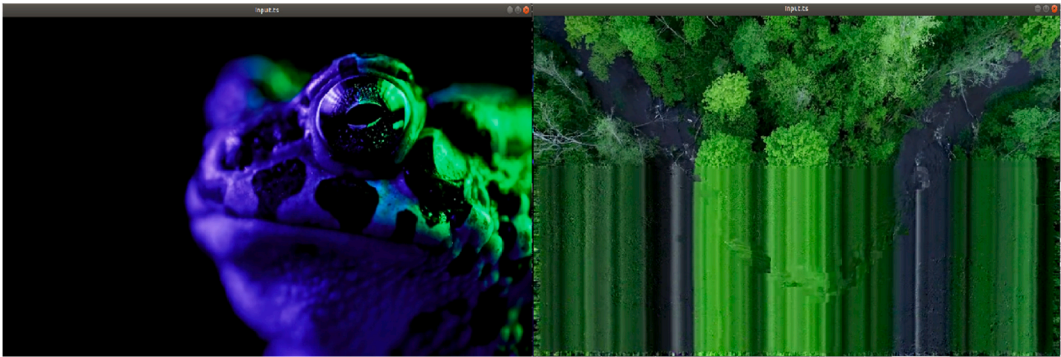


Fig. 3. Normal and corrupted frames in the video obtained after the transfer.

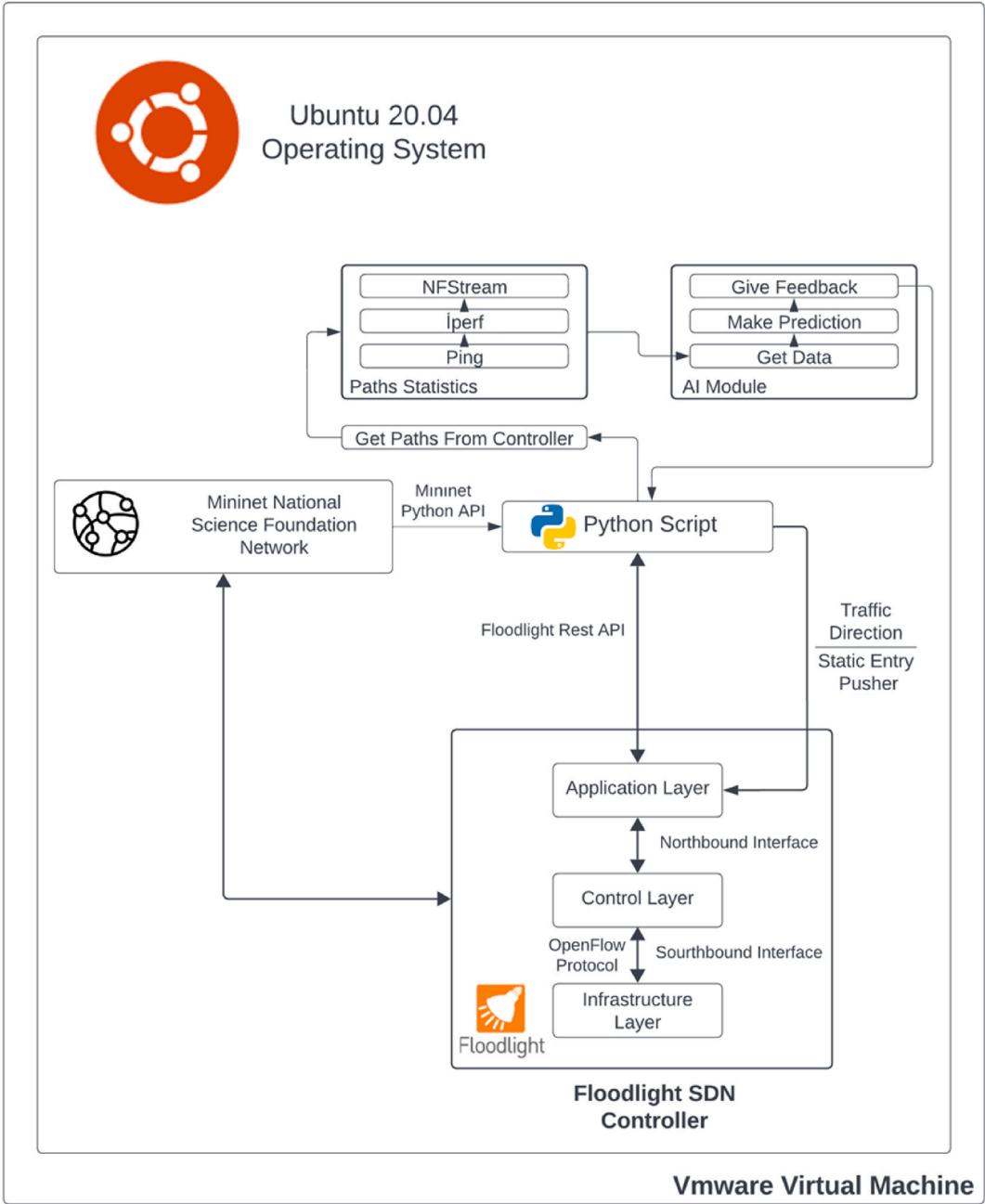


Fig. 4. Proposed system flow diagram.

traffic levels. Two different traffic levels are created for each route, and data transmission is made according to these traffic levels. The big picture of the network for scenarios is constructed, and the devices are shown in Fig. 1 in the Introduction section. The network topology consists of 14 switches, one client, and one server, forming a structured environment for data transmission and analysis. There is also a Floodlight controller that controls all switches on the network. These network devices serve as the foundation for the simulated scenarios. While designing scenarios, different clients are added to the network, and traffic is created between them.

• Video Transfer - First Scenario

The first scenario is based on the shortest path between the client and the server. The shortest route on the mentioned network follows the switches 0–3–8–9 and reaches the destination with 3 hops. The first scenario is presented in Fig. 5.

In the scenario, 2 different traffic levels are created between the receiver and the sender, and 4 clients are assigned to switches 3 and 8. In a low-traffic environment, 4 selected clients establish 2 connections between each other, creating TCP traffic and using up to 30 % of the bandwidth. In a high-traffic environment, 8 clients create TCP traffic by establishing 4 connections using approximately 90 % of the bandwidth. Once traffic is generated, the video is transferred between the main client and the server. Details on client configurations for generating low and high traffic levels are given in Tables 3 and 4.

• Video Transfer - Second Scenario

In the second scenario, the number of hops is increased by one, and a path of 4 hop length is preferred. Video transfer between client and server follows the switches 0–1–7–10–9. Two different traffic levels are created on this path and the visualization of the scenario is shown in Fig. 6.

In the second scenario, 4 clients are assigned to switches 1 and 10. In a low-traffic environment, it creates TCP traffic by establishing 2 connections between 4 clients, similar to the first scenario, and uses up to 30 % of the bandwidth. In a high-traffic environment, 8 clients create TCP traffic by establishing 4 connections using approximately 90 % of the bandwidth. In both cases, tests are performed by transmitting video.

• Video Transfer - Third Scenario

In the third scenario, the number of hops is increased to a path with 5 hops. Video transfer between the client and the server follows the

Table 3

Example traffic information in a low-traffic environment.

Low Traffic Environment			
SOURCE IP	DESTINATION IP	BANDWIDTH	PROTOCOL
10.0.0.9	10.0.0.4	1500K	TCP
10.0.0.17	10.0.0.15	1500K	TCP

Table 4

Example traffic information in a high-traffic environment.

High Traffic Environment			
SOURCE IP	DESTINATION IP	BANDWIDTH	PROTOCOL
10.0.0.9	10.0.0.4	2250K	TCP
10.0.0.17	10.0.0.15	2250K	TCP
10.0.0.18	10.0.0.16	2250K	TCP
10.0.0.21	10.0.0.19	2250K	TCP

switches 0–2–5–13–10–9. Similarly, low and high traffic is generated in this scenario using the same number of clients and the same amount of bandwidth. Clients to generate traffic are assigned to switches 2 and 10. The scenario is shown in Fig. 7.

3.3. Complexity analysis

To accurately evaluate the efficiency of the proposed algorithm, it is essential to analyze its computational complexity, particularly in comparison with benchmark routing algorithms. The algorithm takes as input the IPv4 addresses of the client and server and determines the optimal routing path based on traffic predictions generated by a pre-trained AI classification model. The following parameters are defined:

- k : the number of candidate shortest paths between the client and server,
- f : the time complexity of adding and removing flows to/from the flow table,
- s : the time complexity of computing path statistics,
- p : the time complexity of the model's prediction function.

Accordingly, the overall time complexity of the algorithm simplifies $O(k \cdot (f + s + p))$. Assuming that f , s and p are constant or grow modestly with network size, the algorithm remains efficient and suitable for real-time routing decisions.

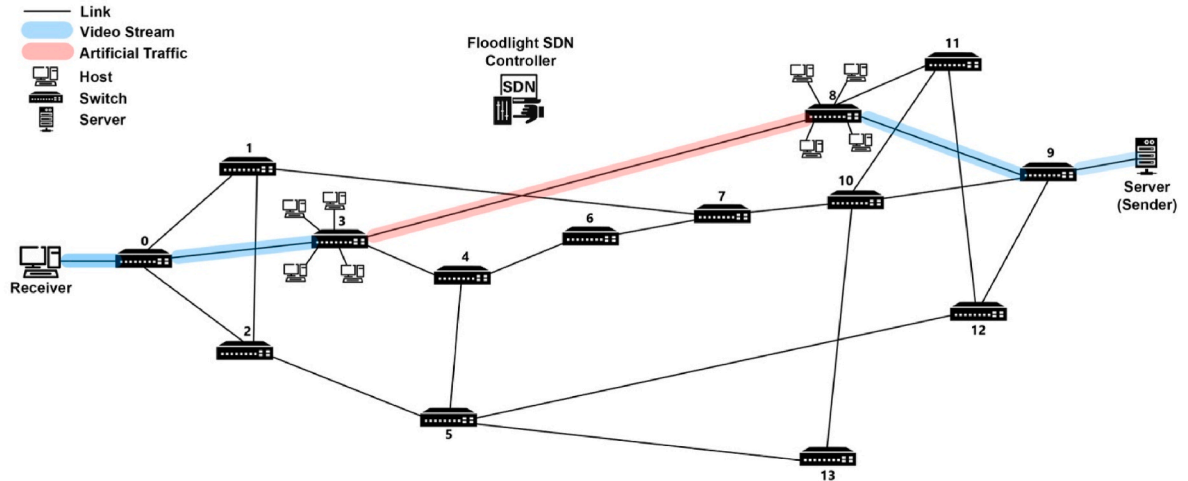


Fig. 5. Visualization of the first video transmission scenario.

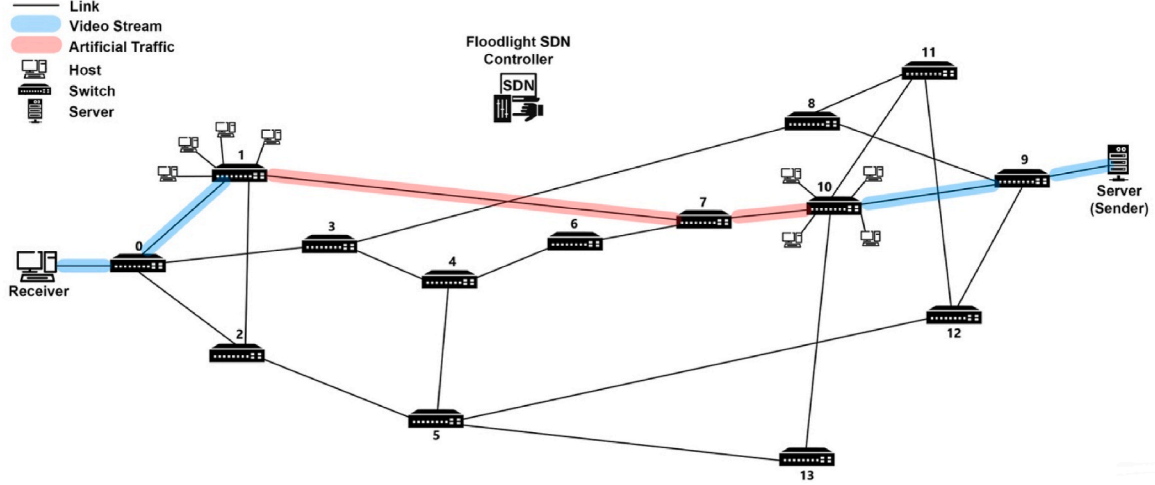


Fig. 6. Visualization of the second video streaming scenario.

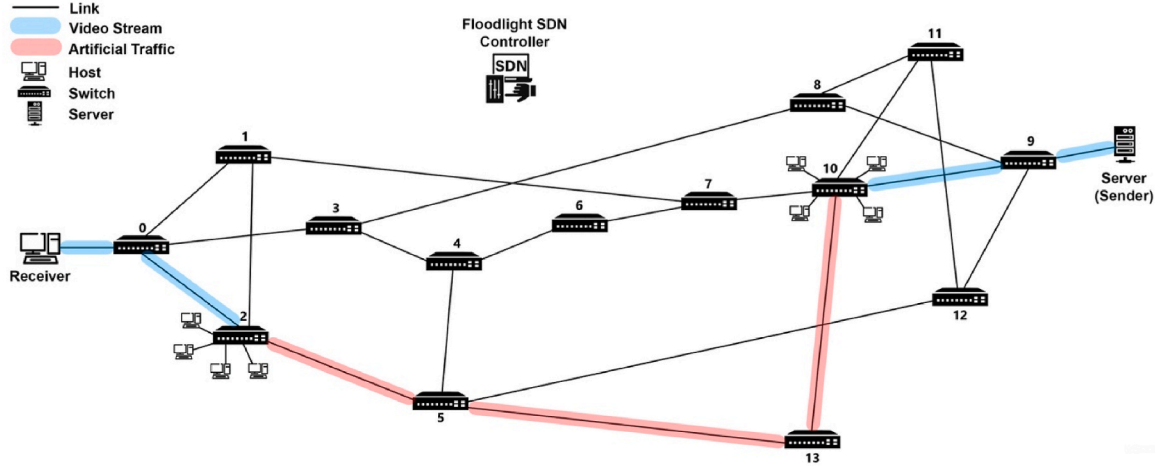


Fig. 7. Visualization of the third video transmission scenario.

3.4. Measuring video quality after video transfer

Video is transferred between the client and the server based on the scenarios. The transferred video is 66 s long and has a resolution of 1920x1080 pixels. The PSNR and SSIM parameters are employed to measure the effects of the existing traffic on the transmission path and the video transmission protocol (TCP or UDP) on video quality. These metrics play a crucial role in evaluating the quality of the transmitted video compared to the original. PSNR and SSIM are considered key metrics for assessing transmission performance. To these metrics, the impacts of different traffic levels and protocols on video quality can be analyzed with concrete data [41].

PSNR is a commonly used metric to evaluate video quality. This metric measures how successful the processed version of an original video is against processes such as format changes, compression, or transmission over the network. To calculate PSNR, the first step is to calculate MSE (Mean Square Error) (E.1.1), the average of the squared differences of pixel values in the referenced original video and the processed video frames. Then, using this MSE value, video quality is measured with the PSNR (E.1.2) Equation.

$$MSE = \frac{1}{m \times n} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} [O(i,j) - D(i,j)]^2 \quad (E.1.1)$$

$$\begin{aligned} PSNR &= 10 \cdot \log_{10} \left(\frac{MAX_1^2}{MSE} \right) \\ &= 20 \cdot \log_{10} \left(\frac{MAX_1}{\sqrt{MSE}} \right) \\ &= 20 \cdot \log_{10} \left(\frac{MAX_1}{\sqrt{MSE}} \right) \end{aligned} \quad (E.1.2)$$

In Equation (E.1.1), m represents the number of rows and n represents the number of columns in the image matrix. $O(i,j)$ denotes the original pixel value at position (i,j) , while $D(i,j)$ represents the corresponding pixel value in the distorted or processed image. In Equation (E.1.2), MAX represents the peak value of pixels and is generally considered 255 when pixels are expressed as 8 bits. The equation expressing these calculations is (E.1.3):

$$PSNR = 48.1308036087 - 20 \cdot \log_{10} \left(\frac{MAX_1}{\sqrt{MSE}} \right) \quad (E.1.3)$$

SSIM is a metric that compares the similarity between two images. SSIM aims to obtain results closer to the human visual system and includes measurements such as structural, brightness, and contrast similarity between images. It takes values between -1 and 1 . Values approaching 1 indicate high similarity between two images, while

values approaching -1 indicate low similarity. The equation expressing these calculations is (E.1.4):

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (E.1.4)$$

- x and y are the two images being compared.
- μ_x and μ_y are the average values of the pixel values of the images.
- σ_x and σ_y are the standard deviations of the pixel values.
- σ_{xy} is the covariance between x and y .
- C_1 and C_2 are small constant values for stability and normalization.

Information about the transmission path between the client and the server is obtained using different tools (iperf3, NFStream, ping). A total of 49 features are collected regarding the transmission path using the aforementioned tools. However, not all of these features is directly related to the traffic level on the path. Through conducted analyses, the features influencing the traffic level have been identified and provided as input to the machine learning algorithm employed in this study. The results of these analyses reveal the effects related to traffic levels both before and after the transfer and are detailed comprehensively in the conclusion section of the study. The tools used to obtain this data, along with their names and descriptions, are presented in the Supplementary File.

3.5. Generating data with the scenarios

Data is produced with scenarios designed in the Mininet environment. A program was written in Python to generate data automatically. Mininet and other tools support Python because it's easy to use. The program flowchart is summarized in Fig. 8. The network topology is created, Floodlight controller configuration settings are made, and the paths for the scenario are set with Floodlight StaticEntryPusher. Then, traffic is created with iperf3, the parameters are obtained with the previously mentioned tools, and video transfer is done and recorded between the client and server. Video is tested before and after transmission, and finally, the system is reset and run cyclically. Threads are created multiple times within the program to perform multiple operations.

4. Performance results

In the simulation environment, a total of 867 records are generated across three different scenarios. The data generation process is conducted in two phases for each scenario, representing low and high traffic conditions. Each record contains 50 features, with 49 representing various metrics collected during and before the video transmission, and the last feature (label) indicating the traffic type in the environment. The data collection process is carried out using NFStream, iperf3, and ping tools to measure the connection quality between two devices in the network. These tools provide detailed insights into network performance by collecting bi-directional traffic statistics such as average round-trip time (RTT), packet loss, bits per second (throughput), bandwidth utilization rate, and other relevant metrics. The distribution of data based on traffic levels and the number of hops in each scenario is illustrated in Fig. 9.

The size of the video transferred in high and low-traffic environments varies, and data loss occurs. The original video size used during data generation is 39478308 bits. When the video is transferred in a low-traffic environment, the average transferred file size is 39320447.71 bits, and 157860.28 bits are lost. It is observed that there is approximately 0.4 % data loss. When the video is transferred in a high-traffic environment, the average transferred file size is 36388202.41 bits, and 3090105.58 losses occur. It is observed that there is approximately

7.82 % data loss.

Upon analyzing the effects of traffic type on different features, it is observed that traffic type has a direct impact on 19 features. During the analysis process, the correlation values between these features and the traffic type are examined. Based on the obtained correlation values, the features with the highest correlation to traffic type are identified as average RTT, packet loss, bandwidth utilization rate, and bits per second (throughput), respectively. In line with these analysis results, the correlation matrix illustrating the relationship between traffic type and the 19 features is presented in Fig. 10.

To further examine the distribution of the four features that show the highest correlation with traffic type and their relationships with traffic conditions, box plots for these features are presented in Fig. 11. The results indicate that the average RTT in low-traffic conditions is 306.27 ms, while this value rises to 1710.59 ms in high-traffic conditions, demonstrating a significant increase in network latency under heavy traffic.

Similarly, packet loss is observed as 0 % in low-traffic environments, whereas it reaches 13.99 % under high-traffic conditions, indicating a substantial increase in transmission losses with higher traffic levels. The bandwidth utilization ratio is detected to be 33.18 % on average in low-traffic environments; it increases to 93.31 % under high-traffic conditions, revealing that the network's bandwidth becomes heavily saturated under these circumstances.

Lastly, the number of bits transmitted per second averages 6.68 Mbps in low-traffic environments, while this value drops to 0.67 Mbps in high-traffic conditions. This finding illustrates a significant reduction in data transmission speed under heavy traffic. The box plots presented in Fig. 11 visually depict the impact of traffic levels on specific features and allow for a detailed examination of how the data varies under different traffic conditions.

PSNR and SSIM parameters are used to measure the effect of the TCP traffic constructed in the scenario on the transmitted video quality. These parameters give us information about how different the video is from the original video. High values indicate that the video quality is close to the original, while low values indicate distortion in the video frames. Upon examining the data, the average PSNR value for videos transmitted in low-traffic environments is calculated to be 36.45 dB. This result indicates that the video quality is quite close to the original. In contrast, the average PSNR value drops to 21.97 dB in high-traffic environments, signifying a noticeable decline in video quality. In terms of SSIM, the average SSIM for videos transmitted in low-traffic environments is initially measured at 0.996, indicating that the video quality is highly similar to the original. However, in high-traffic environments, this value decreases to 0.917, indicating a significant reduction in the structural similarity of the video. These results reveal that traffic levels have a significant impact on video quality, with higher quality video transmission occurring under low-traffic conditions, whereas quality degradation increases under high-traffic conditions. In the collected data, the average PSNR and SSIM values of the transmitted videos in low and high-traffic environments are shown in Fig. 12.

Data is collected using NFStream, iperf3, and ping tools to measure the connection quality between two devices in the dataset. When the collected data is examined, it is observed that all features do not directly affect the result value, and features that do not have an effect are removed from the dataset. A total of 33 features were obtained with the NFStream tool. The traffic level of these features was examined by creating a correlation matrix, and it was observed that 12 features affected the result feature. Features that have no effect are generally related to packet sizes. When 12 features collected with other tools are examined, it is observed that 7 features affect the traffic level. As a result, the number of features, which was initially 49, was reduced to 19 by feature extraction as a result of data analysis. These 19 features constitute the key inputs for training the binary classification model. These selected features are carefully chosen to enhance the model's accuracy and reduce noise that could be introduced by irrelevant data.

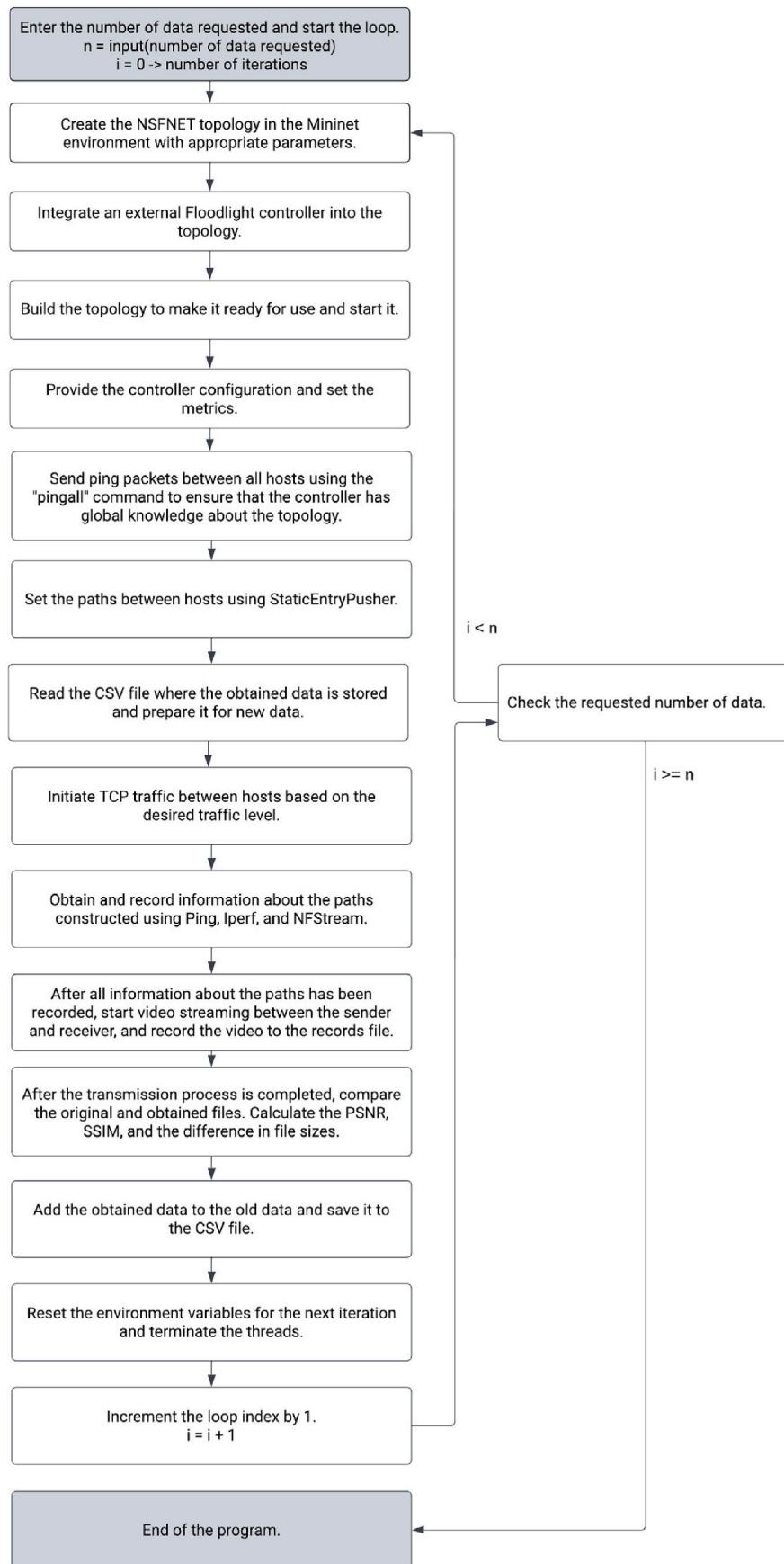


Fig. 8. Data generation process flowchart.

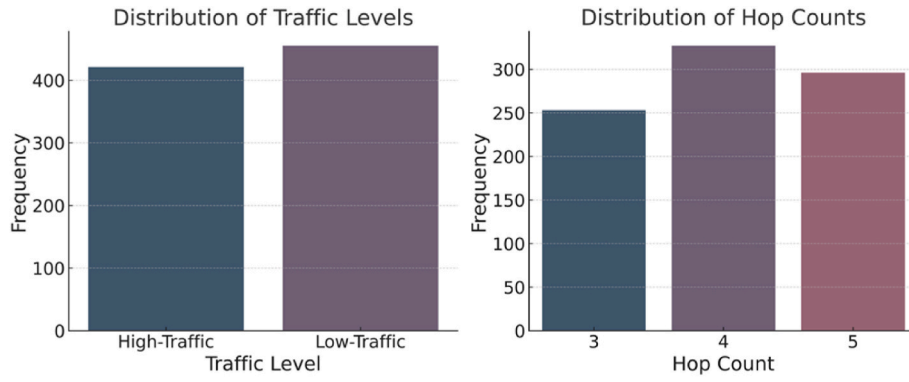


Fig. 9. Distribution of data based on traffic level and hop count.

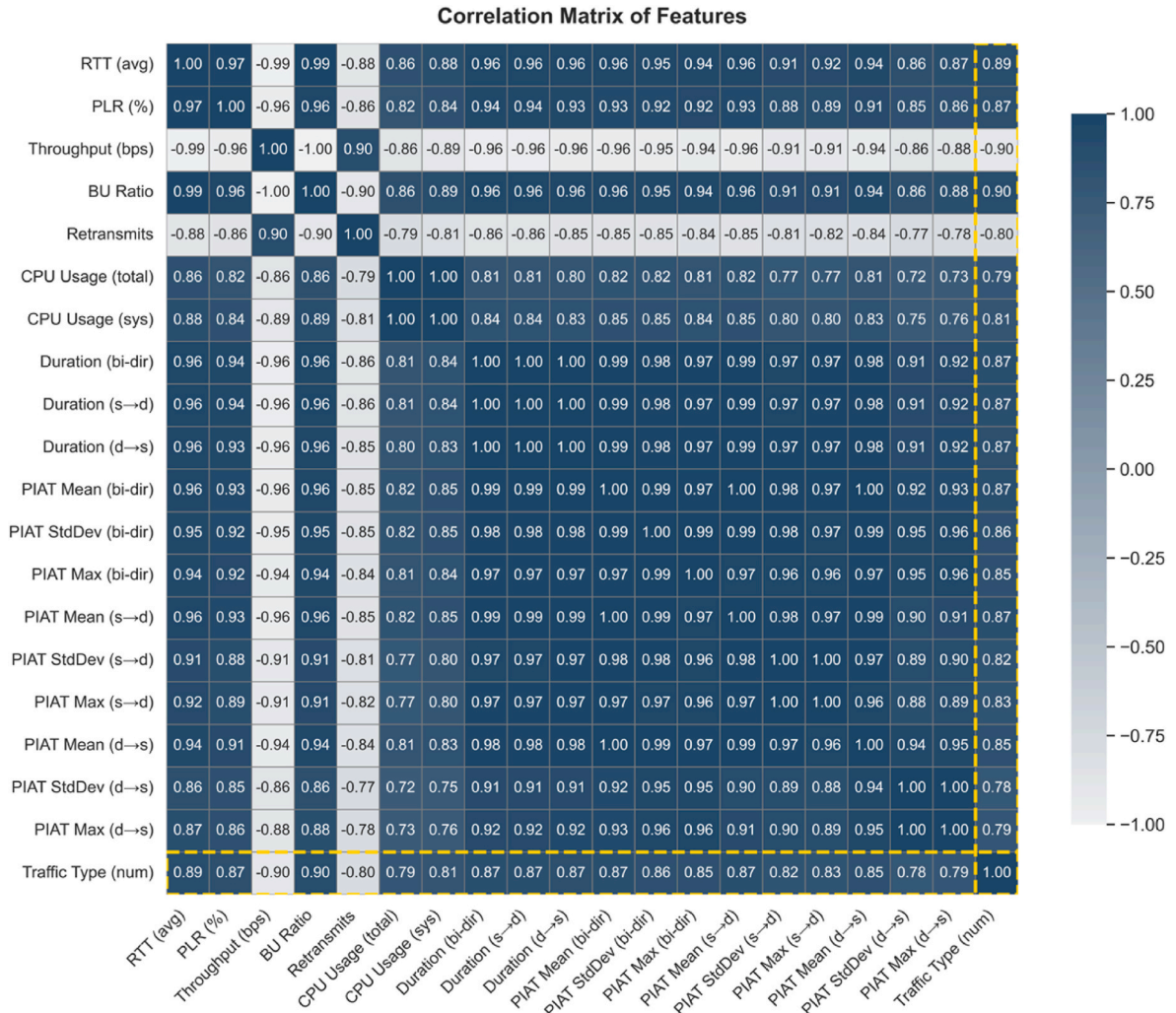


Fig. 10. Correlation matrix illustrating the correlations among the 19 different features in the dataset.

Thus, it is aimed to optimize the model performance and provide faster and more effective classification.

4.1. Binary classification training results

After the analysis, the dataset is trained using various binary classification models. During the training process, the dataset is split into 70 % for training, 15 % for validation, and 15 % for testing. Experiments are conducted using XGBoost, Random Forest, SVM, KNN, AdaBoost, and

Logistic Regression models. According to the results, the overall performance of the models is quite high due to the dataset's structure and the clear distinction between features. The outcomes based on F1 score, precision, and recall metrics are presented in Table 5. These results demonstrate the overall performance of the models and their effects on the dataset. Logistic Regression achieves the highest performance in terms of the F1 score, reaching 96.4 %. This result demonstrates its superiority over other models by maintaining a strong balance between precision and recall. The high precision indicates the model's accuracy

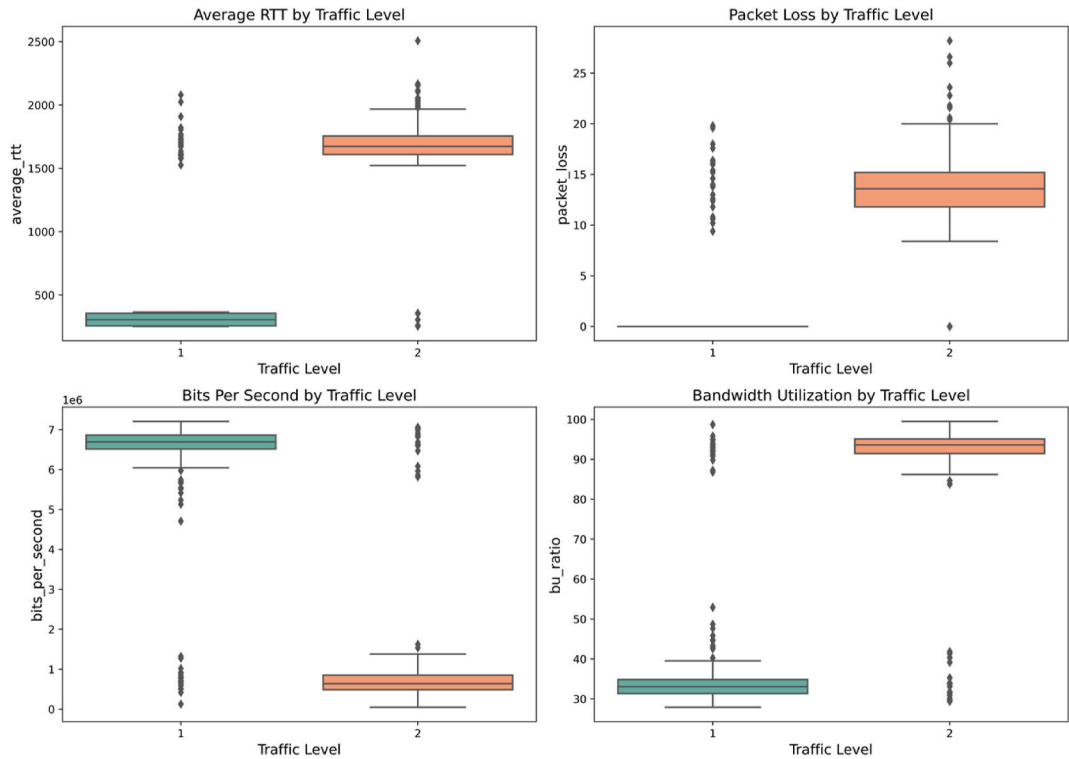


Fig. 11. Box plots show the relationship between traffic type and the features of average RTT, packet loss, bandwidth utilization rate, and bits per second.



Fig. 12. PSNR and SSIM values based on traffic level.

Table 5
Performance results of machine learning models.

Model	Precision	Recall	F1 Score
XGBoost	0.956	0.956	0.956
Random Forest	0.943	0.971	0.957
SVM	0.955	0.941	0.948
knn	0.917	0.971	0.943
AdaBoost	0.929	0.956	0.942
Logistic Regression	0.957	0.971	0.964

in predicting the positive class, while the strong recall suggests its effectiveness in minimizing false negatives.

The success of the Logistic Regression model is primarily due to the compatibility of the dataset's structure with this model. The linear separability of features within the dataset enables the Logistic Regression model to effectively learn these features and utilize them efficiently

during the classification process. This phenomenon is observed in the confusion matrix of the Logistic Regression model. The confusion matrix presented in Table 6 provides a detailed insight into the classification performance of the model.

The ROC (Receiver Operating Characteristic) curve of the Logistic Regression model is a crucial visualization that evaluates the model's ability to distinguish between positive and negative classes. The ROC curve presented in Fig. 13 confirms the overall performance of the

Table 6
Confusion matrix of logistic regression model.

		Predicted Class	
		Low Traffic	High Traffic
Actual Class	Low Traffic	61	3
	High Traffic	2	66

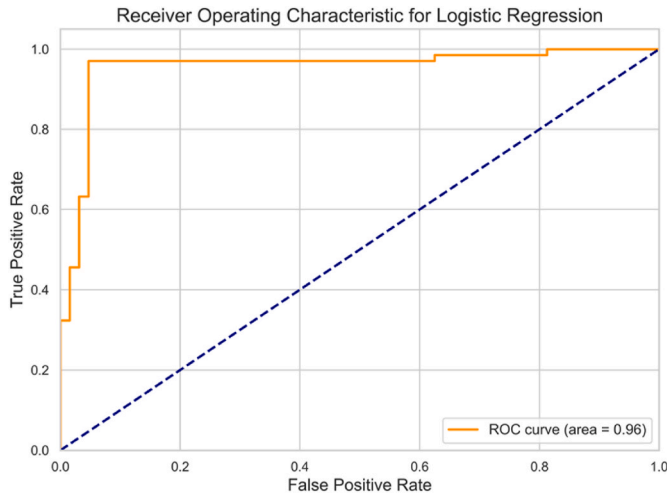


Fig. 13. ROC curve for the Logistic Regression model.

model, resulting in a high AUC (Area Under the Curve) value of 0.955. This indicates that the Logistic Regression model is highly effective in distinguishing the positive class.

In conclusion, the Logistic Regression model demonstrates the highest performance among the binary classification models used in this study. Although other models also performed well, the Logistic Regression model maximizes overall classification success, particularly by offering a balanced precision and recall ratio. The structure of the dataset and the low noise level significantly contribute to this success. Synthetic data generation can involve less noise compared to real-world data, which generally enhances model performance. Moreover, during synthetic data generation, it is possible to create meaningful and distinct separations within the dataset, allowing linear models like Logistic Regression to learn these separations more effectively.

4.2. Performance of the proposed AI-based routing algorithm

The proposed AI-based routing algorithm is tested in a scenario based on the NSFNET topology. In the scenario, there are 19 clients generating traffic, 1 main client (receiving the video), and 1 server (sending the video). In addition, the Floodlight controller, which we describe as the brain of the network, is set to the hop count value as a metric. Fig. 14 shows the traffic flows and degrees between hosts and provides a general visualization of the scenario.

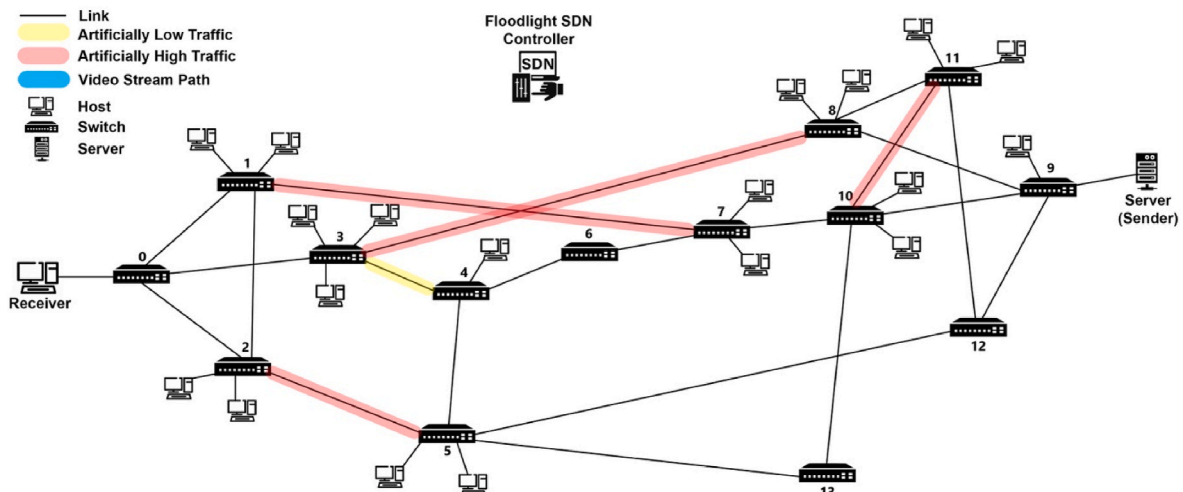


Fig. 14. Visualization of the scenario designed to test the performance of the proposed algorithm.

During the testing phase, data is transferred between the client and the server without any intervention in the network first. The Floodlight controller performs data flow over the shortest distance with the hop count metric and follows the switches 0-3-8-9. Heavy traffic on this path negatively affects data flow and data losses occur.

During the testing phase, data is transferred between the client and the server without any intervention in the network first. The Floodlight controller performs data flow over the shortest distance using the hop count metric and follows the switches 0-3-8-9. Heavy traffic on this path negatively affects data flow and causes data losses. In previous tests, quality values were low when the video was transmitted along the path selected by the controller.

In addition to the default hop-count-based routing, a QoS-aware routing method—designed to consider basic performance metrics such as latency and bandwidth—was also evaluated. However, similar to the hop-count approach, this method failed to adequately adapt to dynamic traffic conditions, leading to suboptimal routing decisions and degraded video quality.

In the second part of the testing phase, the proposed routing algorithm is analyzed. The same environment variables as in the first stage are used for testing. Before the transfer between the client and the server, the proposed algorithm runs and finds the route with a low traffic level. During the transfer, it follows the switches 0-3-4-5-12-9. The lower traffic on this path allows for more successful data transfer. Fig. 15 shows the path that the proposed algorithm follows during data transfer.

After executing the scenarios of all three routing approaches, the data collected during the tests are compared. These include the default hop-count-based routing and the QoS-aware routing—both provided by the Floodlight controller—as well as the proposed AI-based routing algorithm. For comparison, the network is monitored for 60 s, and RTT and throughput values are recorded along each selected path. A comparison of the recorded data is illustrated in line graphs in Figs. 16 and 17.

In the first test phase, as a result of the default Floodlight routing algorithm with the hop count metric, an average RTT of 2063.14 ms and a throughput of 2135.90 bps are obtained. In the second phase, using the QoS-aware routing method of the Floodlight controller, RTT is reduced to 1039.25 ms, and throughput is measured at 861.56 bps. In the third phase, with the proposed AI-based algorithm, these metrics are further improved, achieving 354.05 ms RTT and 6313.50 bps throughput. As a result of the comparison, the proposed algorithm successfully detects the path with the lowest traffic levels and ensures more efficient data transfer.

These findings demonstrate that the proposed algorithm significantly reduces RTT and meaningfully increases throughput compared to both

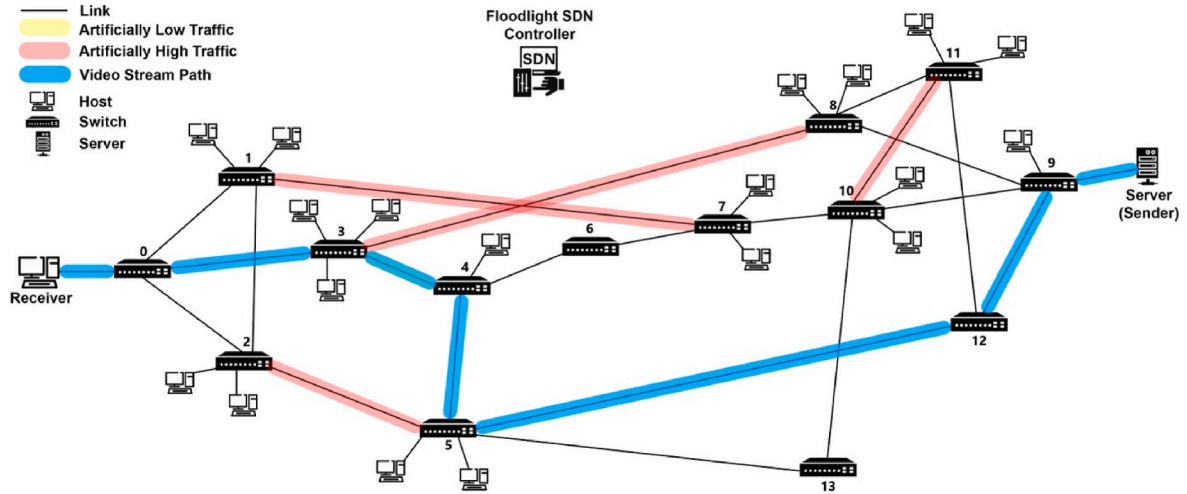


Fig. 15. Visualization of the path followed by the proposed algorithm during data transfer.

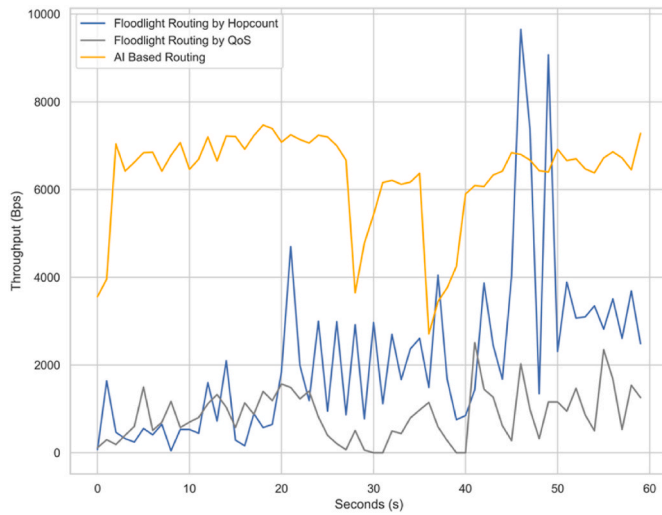


Fig. 16. Comparison of Throughput between traditional Floodlight, QoS-aware and proposed routing.

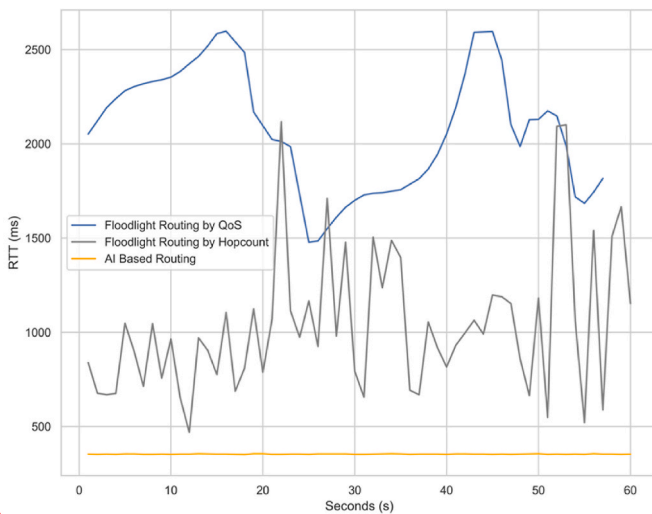


Fig. 17. Comparison of RTT between traditional Floodlight, QoS-aware and proposed routing.

the default and QoS-aware routing methods. Additionally, the results indicate that the proposed algorithm more accurately identifies low-traffic paths in the network, allowing for faster and more reliable data transmission. The marked reduction in RTT reflects minimized network latency, while the substantial increase in throughput confirms optimized data transmission capacity and higher operational efficiency.

A more comprehensive evaluation of the results is also conducted in terms of video quality. Data transfers using the default algorithm yielded PSNR and SSIM values of 21.45 dB and 0.927, respectively. With QoS-aware routing, these values slightly improved to 21.57 dB for PSNR and 0.929 for SSIM. However, the proposed AI-based routing algorithm significantly enhanced video quality, reaching 37.27 dB PSNR and 0.997 SSIM. These results confirm that the proposed algorithm not only improves network performance but also delivers a markedly better user experience. The substantial improvements in PSNR and SSIM values validate the effectiveness of the AI-based approach in maintaining video integrity even under varying traffic conditions. In conclusion, the proposed AI-based routing algorithm outperforms both the default and QoS-aware routing methods in reducing latency, increasing data transfer rates, and improving video quality.

To further assess the real-world applicability of the proposed algorithm, its resource consumption was evaluated on a Raspberry Pi 5 with 8 GB RAM. During a continuous 12-min and 54-s recording session, the RAM usage remained relatively stable around 50 %, indicating efficient memory management. However, CPU usage exhibited significant fluctuations between 10 % and 90 %, reflecting the algorithm's dynamic nature and real-time decision-making processes. The resource usage observed during this test is illustrated in Fig. 18, which displays CPU and RAM utilization over time during algorithm execution on the Raspberry Pi 5. These results demonstrate that the proposed algorithm is capable of running on resource-constrained edge devices, albeit with fluctuating CPU demands that should be considered for future scalability and performance tuning.

5. Discussion

This study explores the potential and advantages of an AI-based dynamic routing algorithm in an SDN environment. The findings demonstrate that the proposed algorithm significantly improves network performance and user experience compared to traditional hop-count-based routing and the QoS-aware routing approach provided by the Floodlight controller. These improvements become more pronounced in high-traffic scenarios, highlighting the study's relevance to real-world networks.

Additionally, this research contributes to the existing literature on

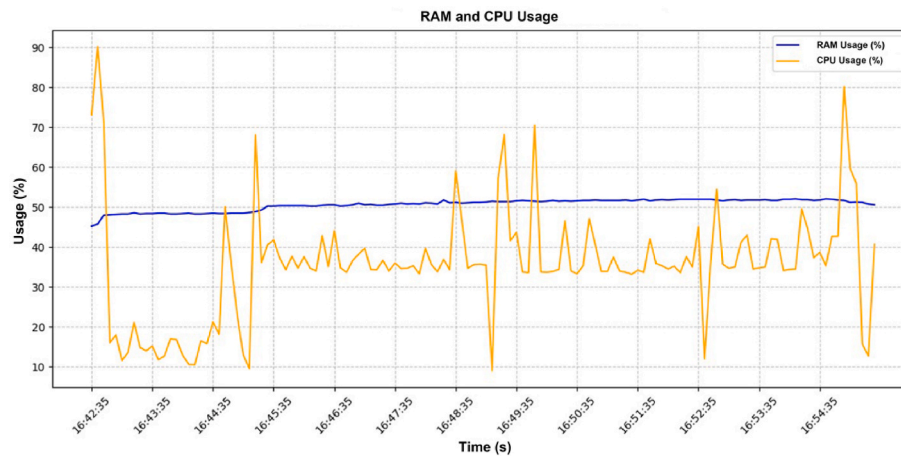


Fig. 18. CPU and RAM utilization on Raspberry Pi 5 (8 GB) during AI-based routing execution.

SDN and AI integration in several ways. First, it proposes a practical AI-based routing algorithm that dynamically evaluates traffic levels to make routing decisions. Although AI-based routing has been widely discussed in the literature, this study provides a concrete implementation in a widely used SDN simulation environment (Mininet) with realistic scenarios using the NSFNET topology.

Second, the study demonstrates the effectiveness of machine learning models—particularly Logistic Regression—in classifying network traffic levels. Creating a unique dataset and a detailed feature analysis offer a valuable resource for future research. Identifying 19 key features affecting traffic levels can facilitate the development of more focused and efficient modeling approaches in subsequent studies.

Third, a detailed analysis of the impact of traffic on video quality (measured using PSNR and SSIM metrics) underscores the importance of AI-based routing for multimedia applications. The empirical evidence of video quality degradation under high traffic conditions serves as a motivation for developing QoS-focused network management strategies. Several avenues for future research emerge from this study:

- Evaluating the proposed algorithm in larger and more complex network topologies, as well as under different traffic types (e.g., UDP).
- Testing the algorithm in real-world network environments and addressing practical challenges such as controller load, real-time data collection, and processing.
- Exploring the applicability of different machine learning and deep learning techniques for traffic classification and routing decisions.
- Developing integrated AI-based network management solutions that also consider other critical factors, such as network security and energy efficiency.
- Conducting a more detailed analysis of SDN controller resource consumption (CPU, memory, etc.) and developing strategies to address scalability issues.
- Investigating the feasibility and benefits of AI-based routing algorithms in CDN (Content Delivery Network) environments.

5.1. Implications for practical applications

The findings of this study offer valuable insights for network administrators and SDN solution developers. AI-based routing algorithms have demonstrated a significant potential to enhance the efficiency of existing networks and improve user experience. Given the widespread use of high-bandwidth applications such as video streaming, video conferencing, and online gaming, these algorithms could serve as effective tools for optimizing network performance and preventing

congestion.

The proposed approach enables more intelligent routing decisions by considering real-time network conditions using multiple metrics (e.g., RTT, packet loss, bandwidth utilization). This can lead to more efficient resource allocation, reduced latency, and overall improved network performance. Additionally, the relatively low computational cost of the Logistic Regression model facilitates the integration of such AI algorithms into real-time network management systems. However, several factors must be considered for the successful real-world deployment of AI-based routing systems:

- Real-time and reliable data collection: Effective mechanisms must be established to continuously feed AI models with up-to-date network data.
- Sufficient and representative datasets: The dataset used in this study was generated in a simulated environment. Models trained on real-world network data may exhibit different performance characteristics.
- Model interpretability and reliability: Understanding the rationale behind routing decisions and identifying potential errors is crucial for trust and transparency.
- SDN controller performance: The capacity of the SDN controller to run AI algorithms efficiently may pose a limitation in real-world applications. Managing the additional load on the controller and developing scalable solutions will be necessary.

5.2. Limitations of the study

The experiments were conducted in the Mininet virtual network environment, which may not fully capture the complexity and dynamics of real-world networks. The absence of physical distances and reliance on simulated traffic conditions may limit the direct applicability of the results. The dataset used to train the AI model was artificially generated within the Mininet environment. Real-world network traffic can be more diverse and unpredictable, potentially affecting model performance. The study utilized the NSFNET topology. The proposed algorithm's performance may vary in different network architectures, such as data center networks or wireless networks. The study assumes a secure SDN architecture, where communication between the controller and switches is reliable. Potential security threats to the controller and their impact on routing decisions were not addressed. The simulations primarily focused on TCP traffic. Future research could explore the impact of different traffic types (e.g., UDP, ICMP) and their combinations on routing performance.

6. Conclusion

In this study, the impact of an AI-based routing algorithm on network performance in a multi-metric SDN architecture is examined, with a particular focus on its advantages over traditional routing algorithms. SDN controllers (such as Floodlight, ONOS, ODL, and Ryu) typically perform routing based on metrics such as hop count, link status, and bandwidth. However, by default, controllers predominantly determine the shortest path using the hop count metric. Incorporating multiple metrics into routing algorithms introduces the necessity for optimization, making the integration of artificial intelligence into controllers and routing algorithms inevitable. The analysis demonstrates that the AI-based routing algorithm outperforms traditional approaches, including both hop-count and QoS-aware routing methods, leading to enhanced network performance. In the simulations, video transmission was carried out between a selected client and server over the NSFNET topology. During transmission, network traffic influenced certain paths, adversely affecting video quality. Under network conditions that degrade performance, traditional routing and AI-based routing algorithms were compared. During the design phase of the proposed algorithm, various parameters were examined to detect network traffic, and the most relevant parameters were identified through correlation matrix analysis. These parameters help to provide a clearer and more comprehensive understanding of network traffic conditions. The primary objective is to monitor traffic levels using multiple criteria and accurately detect both normal and anomalous conditions. At this stage, artificial intelligence and modern computing capabilities play a crucial role. Ultimately, the findings of this study can serve as a guide for future research on routing and AI-driven optimization in SDN. With SDN's innovative approach, developers gain greater control over network operations, facilitating seamless network management and development.

7. Future works

Although the proposed AI-based routing algorithm demonstrates significant improvements in RTT, throughput, and video quality, several directions remain open for future investigation. First, extending the model to support dynamic and large-scale network environments with real-time adaptability would increase its applicability in practical deployments. Additionally, incorporating reinforcement learning techniques could further optimize routing decisions based on evolving network conditions. Also, real-world scenarios and datasets will contribute to more realistic performance evaluations in the literature. The results of AI-based routing algorithms should also be examined on controllers such as ONOS and OpenDayLight.

CRedit authorship contribution statement

Anıl Dursun İpek: Writing – review & editing, Writing – original draft, Visualization. **Murtaza Cicioğlu:** Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Funding acquisition. **Ali Çalhan:** Writing – review & editing, Writing – original draft, Visualization, Methodology.

Funding

The research leading to these results received funding from Scientific Research Unit of Bursa Uludağ University under Grant Agreement No FGA-2023-1557.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Supplementary data

Supplementary data to this article can be found online at <https://doi.org/10.1016/j.comcom.2025.108222>.

Data availability

<https://mc.manuscriptcentral.com/tomm>

References

- [1] Sandvine The App QoE Company. Global Internet Phenomena Report.3, 2024, 2024, pp. 1–14.
- [2] D. Kreutz, F.M.V. Ramos, P.E. Verissimo, C.E. Rothenberg, S. Azodolmolky, S. Uhlig, Software-defined networking: a comprehensive survey, *Proc. IEEE* 103 (2015) 14–76, <https://doi.org/10.1109/JPROC.2014.2371999>.
- [3] D. Kreutz, F.M.V. Ramos, P. Verissimo, Towards secure and dependable software-defined networks, in: *Proceedings of the Second ACM SIGCOMM Workshop on Hot Topics in Software Defined Networking*, 2013, pp. 55–60, <https://doi.org/10.1145/2491185.2491199>.
- [4] M. Cicioğlu, A. Çalhan, Energy-efficient and SDN-enabled routing algorithm for wireless body area networks, *Comput. Commun.* 160 (2020) 228–239, <https://doi.org/10.1016/j.comcom.2020.06.003>.
- [5] N. Feamster, J. Rexford, E. Zegura, The road to SDN: an intellectual history of programmable networks, *ACM Sigcomm Computer Communication* 44 (2014) 87–98, <https://doi.org/10.1145/2602204.2602219>.
- [6] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, J. Turner, OpenFlow: enabling innovation in campus networks, *Comput. Commun. Rev.* 38 (2008) 69, <https://doi.org/10.1145/1355734.1355746>.
- [7] I.F. Akyildiz, A. Lee, P. Wang, M. Luo, W. Chou, A roadmap for traffic engineering in SDN-OpenFlow networks, *Comput. Netw.* 71 (2014) 1–30, <https://doi.org/10.1016/j.comnet.2014.06.002>.
- [8] F. Hu, Q. Hao, K. Bao, A survey on software defined networking (SDN) and OpenFlow: from concept to implementation, *IEEE Communications Surveys & Tutorials* (2014), <https://doi.org/10.1109/COMST.2014.2326417>.
- [9] F. Hu, Q. Hao, K. Bao, A survey on software-defined network and OpenFlow: from concept to implementation, *IEEE Communications Surveys and Tutorials* 16 (2014), <https://doi.org/10.1109/COMST.2014.2326417>.
- [10] B.A.A. Nunes, M. Mendonca, X.N. Nguyen, K. Obraczka, T. Turletti, A survey of software-defined networking: past, present, and future of programmable networks, *IEEE Communications Surveys and Tutorials* 16 (2014) 1617–1634, <https://doi.org/10.1109/SURV.2014.012214.00180>.
- [11] M. Cicioğlu, A. Çalhan, MLAR: machine-learning-assisted centralized link-state routing in software-defined-based wireless networks, *Neural Comput. Appl.* 35 (2023) 5409–5420, <https://doi.org/10.1007/s00521-022-07993-w>.
- [12] M. Cicioğlu, A. Çalhan, A multi-protocol controller deployment in SDN-based IoT architecture, *IEEE Internet Things J.* (2022), <https://doi.org/10.1109/JIOT.2022.3175669>, 1–1.
- [13] T. Jafarian, A. Ghaffari, A. Seyfollahi, B. Arasteh, Detecting and mitigating security anomalies in software-defined networking (SDN) using gradient-boosted trees and Floodlight controller characteristics, *Comput. Stand. Interfac.* 91 (2025) 103871, <https://doi.org/10.1016/j.csi.2024.103871>.
- [14] M. Hamdan, E. Hassan, A. Abdelaziz, A. Elhigazi, B. Mohammed, S. Khan, A. V. Vasilakos, M.N. Marsono, A comprehensive survey of load balancing techniques in software-defined network, *J. Netw. Comput. Appl.* 174 (2021), <https://doi.org/10.1016/j.jnca.2020.102856>.
- [15] C. Yu, C. Lumezanu, A. Sharma, Q. Xu, G. Jiang, H.V. Madhyastha, Software-Defined Latency Monitoring in Data Center Networks (2015) 360–372, https://doi.org/10.1007/978-3-319-15509-8_27.
- [16] A.-C. Anadiotis, L. Galluccio, S. Milardo, G. Morabito, S. Palazzo, SD-WISE: a Software-Defined Wireless Sensor network, *Comput. Netw.* 159 (2019) 84–95, <https://doi.org/10.1016/j.comnet.2019.04.029>.
- [17] B. Xu, X. Chang, D. Xu, S. Wang, U.A. Bhatti, H. Tang, A joint routing and time-slot scheduling load balancing algorithm for in-vehicle TSN, *IEEE Trans. Consum. Electron.* (2025), <https://doi.org/10.1109/TCE.2025.3540890>, 1–1.
- [18] Alpaydin Ethem. *Introduction to Machine Learning*, 4th ed., MIT Press, 2020.
- [19] R. Bozkir, M. Cicioğlu, A. Çalhan, C. Toğay, A new platform for machine-learning-based network traffic classification, *Comput. Commun.* (2023), <https://doi.org/10.1016/j.comcom.2023.05.010>.
- [20] D. Zha, Z.P. Bhat, K.-H. Lai, F. Yang, Z. Jiang, S. Zhong, X. Hu, Data-centric artificial intelligence: a survey, *ACM Comput. Surv.* 57 (2025) 1–42, <https://doi.org/10.1145/3711118>.
- [21] A. Dahal, S. Moulik, R. Mukherjee, FeL-MAR: federated learning based multi resident activity recognition in IoT enabled smart homes, *Future Gener. Comput. Syst.* 163 (2025) 107552, <https://doi.org/10.1016/j.future.2024.107552>.
- [22] S.A. Latif, F.B.X. Wen, C. Iwendi, L.F. Wang, S.M. Mohsin, Z. Han, S.S. Band, AI-empowered, blockchain and SDN integrated security architecture for IoT network of cyber physical systems, *Comput. Commun.* 181 (2022) 274–283, <https://doi.org/10.1016/j.comcom.2021.09.029>.
- [23] M.R. Belgaum, F. Ali, Z. Alansari, S. Musa, M.M. Alam, M.S. Mazliham, Artificial intelligence based reliable load balancing framework in software-defined

- networks, *Comput. Mater. Continua (CMC)* 70 (2021), <https://doi.org/10.32604/cmc.2022.018211>.
- [24] M. Latah, L. Toker, Application of artificial intelligence to software defined networking: a survey, *Indian J. Sci. Technol.* 9 (2016), <https://doi.org/10.17485/ijst/2016/v9i44/89812>.
- [25] Floodlight Floodlight, Floodlight documentation. <https://github.com/floodlight/floodlight>, 2016. (Accessed 20 March 2025).
- [26] Mininet Mininet, An instant virtual network on your laptop (or other PC) - Mininet, Mininet.Org. <https://mininet.org/download/>, 2014. (Accessed 20 March 2025).
- [27] A.D. İpek, B. Arslan, M. Cicioğlu, A. Çalhan, Machine learning-assisted on-demand video streaming in software-defined networks, in: *23rd Academic Informatics Conference*, 2025.
- [28] A. Warsama, Traffic engineering with SDN: Optimising traffic load-balancing with OpenFlow, Dissertation, 2020, pp. 1–44. <https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1449012&dswid=9575> (Accessed 11 March 2024).
- [29] B. Babayigit, B. Ulu, Deep learning for load balancing of SDN-based data center networks, *Int. J. Commun. Syst.* 34 (2021), <https://doi.org/10.1002/dac.4760>.
- [30] A.H. Alhilali, A. Montazerolghaem, Artificial intelligence based load balancing in SDN: a comprehensive survey, *Internet of Things (Netherlands)* 22 (2023), <https://doi.org/10.1016/j.iot.2023.100814>.
- [31] M.I. Hamed, B.M. ElHalawany, M.M. Fouda, A.S.T. Eldien, A new approach for server-based load balancing using software-defined networking, in: *2017 IEEE 8th International Conference on Intelligent Computing and Information Systems, ICICIS 2017*, 2017, <https://doi.org/10.1109/INTELCIS.2017.8260023>.
- [32] E.Ç. Kük, M. Erel-Özçevik, Access protocol aware controller design for eMBB traffic in SD-CDN, *Comput. Netw.* 205 (2022) 108686, <https://doi.org/10.1016/j.comnet.2021.108686>.
- [33] A. Nur Temurçin, M. Ersoy, Investigation of AI-based network traffic management mechanisms in network infrastructure, *International Journal of Sustainable Engineering and Technology* 7 (2023) 31–40.
- [34] P. Bakonyi, T. Boros, I. Kotuliak, Classification based load balancing in content delivery networks, in: *2020 43rd International Conference on Telecommunications and Signal Processing, TSP 2020*, 2020, <https://doi.org/10.1109/TSP49548.2020.9163470>.
- [35] T. Semong, T. Maupong, S. Anokye, K. Kehulakae, S. Dimakatso, G. Boipelo, S. Sarefo, Intelligent load balancing techniques in software defined networks: a survey, *Electronics (Switzerland)* 9 (2020), <https://doi.org/10.3390/electronics9071091>.
- [36] C.X. Cui, Y. Bin Xu, Research on load balance method in SDN, *International Journal of Grid and Distributed Computing* 9 (2016), <https://doi.org/10.14257/ijgdc.2016.9.1.03>.
- [37] S. Degirmenci, D. Yiltas, Software defined network application with artificial intelligence techniques, *Journal of Engineering Sciences and Design* 8 (2020) 999–1009, <https://doi.org/10.21923/jesd.676110>.
- [38] A.A. Gebremariam, M. Usman, M. Qaraqe, Applications of artificial intelligence and machine learning in the area of SDN and NFV: a survey, in: *16th International Multi-Conference on Systems, Signals and Devices, SSD 2019*, 2019, <https://doi.org/10.1109/SSD.2019.8893244>.
- [39] J.D. Rogers, Internetworking and the politics of science: NSFNET in internet history, *Inf. Soc.* 14 (1998), <https://doi.org/10.1080/019722498128836>.
- [40] Y.R. Chen, A. Rezapour, W.G. Tzeng, S.C. Tsai, RL-routing: an SDN routing algorithm based on deep reinforcement learning, *IEEE Trans Netw Sci Eng* 7 (2020), <https://doi.org/10.1109/TNSE.2020.3017751>.
- [41] Z. Aouini, A. Pekar, NFStream: a flexible network data analysis framework, *Comput. Netw.* 204 (2022), <https://doi.org/10.1016/j.comnet.2021.108719>.